

#	Description	Level
4.2.5	Verify that, if the application (backend or frontend) builds and sends requests, it uses validation, sanitization, or other mechanisms to avoid creating URIs (such as for API calls) or HTTP request header fields (such as Authorization or Cookie), which are too long to be accepted by the receiving component. This could cause a denial of service, such as when sending an overly long request (e.g., a long cookie header field), which results in the server always responding with an error status.	3

V4.3 GraphQL

GraphQL is becoming more common as a way of creating data-rich clients that are not tightly coupled to a variety of backend services. This section covers security considerations for GraphQL.

#	Description	Level
4.3.1	Verify that a query allowlist, depth limiting, amount limiting, or query cost analysis is used to prevent GraphQL or data layer expression Denial of Service (DoS) as a result of expensive, nested queries.	2
4.3.2	Verify that GraphQL introspection queries are disabled in the production environment unless the GraphQL API is meant to be used by other parties.	2

V4.4 WebSocket

WebSocket is a communications protocol that provides a simultaneous two-way communication channel over a single TCP connection. It was standardized by the IETF as RFC 6455 in 2011 and is distinct from HTTP, even though it is designed to work over HTTP ports 443 and 80.

This section provides key security requirements to prevent attacks related to communication security and session management that specifically exploit this real-time communication channel.

#	Description	Level
4.4.1	Verify that WebSocket over TLS (WSS) is used for all WebSocket connections.	1
4.4.2	Verify that, during the initial HTTP WebSocket handshake, the Origin header field is checked against a list of origins allowed for the application.	2

#	Description	Level
4.4.3	Verify that, if the application's standard session management cannot be used, dedicated tokens are being used for this, which comply with the relevant Session Management security requirements.	2
4.4.4	Verify that dedicated WebSocket session management tokens are initially obtained or validated through the previously authenticated HTTPS session when transitioning an existing HTTPS session to a WebSocket channel.	2

References

For more information, see also:

- OWASP REST Security Cheat Sheet
- Resources on GraphQL Authorization from graphql.org and Apollo.
- OWASP Web Security Testing Guide: GraphQL Testing
- OWASP Web Security Testing Guide: Testing WebSockets

V5 File Handling

Control Objective

The use of files can present a variety of risks to the application, including denial of service, unauthorized access, and storage exhaustion. This chapter includes requirements to address these risks.

V5.1 File Handling Documentation

This section includes a requirement to document the expected characteristics of files accepted by the application, as a necessary precondition for developing and verifying relevant security checks.

#	Description	Level
5.1.1	Verify that the documentation defines the permitted file types, expected file extensions, and maximum size (including unpacked size) for each upload feature. Additionally, ensure that the documentation specifies how files are made safe for end-users to download and process, such as how the application behaves when a malicious file is detected.	2

V5.2 File Upload and Content

File upload functionality is a primary source of untrusted files. This section outlines the requirements for ensuring that the presence, volume, or content of these files cannot harm the application.

#	Description	Level
5.2.1	Verify that the application will only accept files of a size which it can process without causing a loss of performance or a denial of service attack.	1
5.2.2	Verify that when the application accepts a file, either on its own or within an archive such as a zip file, it checks if the file extension matches an expected file extension and validates that the contents correspond to the type represented by the extension. This includes, but is not limited to, checking the initial 'magic bytes', performing image re-writing, and using specialized libraries for file content validation. For L1, this can focus just on files which are used to make specific business or security decisions. For L2 and up, this must apply to all files being accepted.	1
5.2.3	Verify that the application checks compressed files (e.g., zip, gz, docx, odt) against maximum allowed uncompressed size and against maximum number of files before uncompressing the file.	2
5.2.4	Verify that a file size quota and maximum number of files per user are enforced to ensure that a single user cannot fill up the storage with too many files, or excessively large files.	3
5.2.5	Verify that the application does not allow uploading compressed files containing symlinks unless this is specifically required (in which case it will be necessary to enforce an allowlist of the files that can be symlinked to).	3
5.2.6	Verify that the application rejects uploaded images with a pixel size larger than the maximum allowed, to prevent pixel flood attacks.	3

V5.3 File Storage

This section includes requirements to prevent files from being inappropriately executed after upload, to detect dangerous content, and to avoid untrusted data being used to control where files are being stored.

#	Description	Level
5.3.1	Verify that files uploaded or generated by untrusted input and stored in a public folder, are not executed as server-side program code when accessed directly with an HTTP request.	1
5.3.2	Verify that when the application creates file paths for file operations, instead of user-submitted filenames, it uses internally generated or trusted data, or if user-submitted filenames or file metadata must be used, strict validation and sanitization must be applied. This is to protect against path traversal, local or remote file inclusion (LFI, RFI), and server-side request forgery (SSRF) attacks.	1
5.3.3	Verify that server-side file processing, such as file decompression, ignores user-provided path information to prevent vulnerabilities such as zip slip.	3

V5.4 File Download

This section contains requirements to mitigate risks when serving files to be downloaded, including path traversal and injection attacks. This also includes making sure they don't contain dangerous content.

#	Description	Level
5.4.1	Verify that the application validates or ignores user-submitted filenames, including in a JSON, JSONP, or URL parameter and specifies a filename in the Content-Disposition header field in the response.	2
5.4.2	Verify that file names served (e.g., in HTTP response header fields or email attachments) are encoded or sanitized (e.g., following RFC 6266) to preserve document structure and prevent injection attacks.	2
5.4.3	Verify that files obtained from untrusted sources are scanned by antivirus scanners to prevent serving of known malicious content.	2

References

For more information, see also:

- OWASP File Upload Cheat Sheet
- Example of using symlinks for arbitrary file read
- Explanation of "Magic Bytes" from Wikipedia

V6 Authentication

Control Objective

Authentication is the process of establishing or confirming the authenticity of an individual or device. It involves verifying claims made by a person or about a device, ensuring resistance to impersonation, and preventing the recovery or interception of passwords.

NIST SP 800-63 is a modern, evidence-based standard that is valuable for organizations worldwide, but is particularly relevant to US agencies and those interacting with US agencies.

While many of the requirements in this chapter are based on the second section of the standard (known as NIST SP 800-63B “Digital Identity Guidelines - Authentication and Lifecycle Management”), the chapter focuses on common threats and frequently exploited authentication weaknesses. It does not attempt to comprehensively cover every point in the standard. For cases where full NIST SP 800-63 compliance is necessary, please refer to NIST SP 800-63.

Additionally, NIST SP 800-63 terminology may sometimes differ, and this chapter often uses more commonly understood terminology to improve clarity.

A common feature of more advanced applications is the ability to adapt authentication stages required based on various risk factors. This feature is covered in the “Authorization” chapter, since these mechanisms also need to be considered for authorization decisions.

V6.1 Authentication Documentation

This section contains requirements detailing the authentication documentation that should be maintained for an application. This is crucial for implementing and assessing how the relevant authentication controls should be configured.

#	Description	Level
6.1.1	Verify that application documentation defines how controls such as rate limiting, anti-automation, and adaptive response, are used to defend against attacks such as credential stuffing and password brute force. The documentation must make clear how these controls are configured and prevent malicious account lockout.	1
6.1.2	Verify that a list of context-specific words is documented in order to prevent their use in passwords. The list could include permutations of organization names, product names, system identifiers, project codenames, department or role names, and similar.	2

#	Description	Level
6.1.3	Verify that, if the application includes multiple authentication pathways, these are all documented together with the security controls and authentication strength which must be consistently enforced across them.	2

V6.2 Password Security

Passwords, called “Memorized Secrets” by NIST SP 800-63, include passwords, passphrases, PINs, unlock patterns, and picking the correct kitten or another image element. They are generally considered “something you know” and are often used as a single-factor authentication mechanism.

As such, this section contains requirements for making sure that passwords are created and handled securely. Most of the requirements are L1 as they are most important at that level. From L2 onwards, multi-factor authentication mechanisms are required, where passwords may be one of those factors.

The requirements in this section mostly relate to § 5.1.1.2 of NIST’s Guidance.

#	Description	Level
6.2.1	Verify that user set passwords are at least 8 characters in length although a minimum of 15 characters is strongly recommended.	1
6.2.2	Verify that users can change their password.	1
6.2.3	Verify that password change functionality requires the user’s current and new password.	1
6.2.4	Verify that passwords submitted during account registration or password change are checked against an available set of, at least, the top 3000 passwords which match the application’s password policy, e.g. minimum length.	1
6.2.5	Verify that passwords of any composition can be used, without rules limiting the type of characters permitted. There must be no requirement for a minimum number of upper or lower case characters, numbers, or special characters.	1
6.2.6	Verify that password input fields use type=password to mask the entry. Applications may allow the user to temporarily view the entire masked password, or the last typed character of the password.	1
6.2.7	Verify that “paste” functionality, browser password helpers, and external password managers are permitted.	1

#	Description	Level
6.2.8	Verify that the application verifies the user's password exactly as received from the user, without any modifications such as truncation or case transformation.	1
6.2.9	Verify that passwords of at least 64 characters are permitted.	2
6.2.10	Verify that a user's password stays valid until it is discovered to be compromised or the user rotates it. The application must not require periodic credential rotation.	2
6.2.11	Verify that the documented list of context specific words is used to prevent easy to guess passwords being created.	2
6.2.12	Verify that passwords submitted during account registration or password changes are checked against a set of breached passwords.	2

V6.3 General Authentication Security

This section contains general requirements for the security of authentication mechanisms as well as setting out the different expectations for levels. L2 applications must force the use of multi-factor authentication (MFA). L3 applications must use hardware-based authentication, performed in an attested and trusted execution environment (TEE). This could include device-bound passkeys, eIDAS Level of Assurance (LoA) High enforced authenticators, authenticators with NIST Authenticator Assurance Level 3 (AAL3) assurance, or an equivalent mechanism.

While this is a relatively aggressive stance on MFA, it is critical to raise the bar around this to protect users, and any attempt to relax these requirements should be accompanied by a clear plan on how the risks around authentication will be mitigated, taking into account NIST's guidance and research on the topic.

Note that at the time of release, NIST SP 800-63 considers email as not acceptable as an authentication mechanism (archived copy).

The requirements in this section relate to a variety of sections of NIST's Guidance, including: § 4.2.1, § 4.3.1, § 5.2.2, and § 6.1.2.

#	Description	Level
6.3.1	Verify that controls to prevent attacks such as credential stuffing and password brute force are implemented according to the application's security documentation.	1
6.3.2	Verify that default user accounts (e.g., "root", "admin", or "sa") are not present in the application or are disabled.	1

#	Description	Level
6.3.3	Verify that either a multi-factor authentication mechanism or a combination of single-factor authentication mechanisms, must be used in order to access the application. For L3, one of the factors must be a hardware-based authentication mechanism which provides compromise and impersonation resistance against phishing attacks while verifying the intent to authenticate by requiring a user-initiated action (such as a button press on a FIDO hardware key or a mobile phone). Relaxing any of the considerations in this requirement requires a fully documented rationale and a comprehensive set of mitigating controls.	2
6.3.4	Verify that, if the application includes multiple authentication pathways, there are no undocumented pathways and that security controls and authentication strength are enforced consistently.	2
6.3.5	Verify that users are notified of suspicious authentication attempts (successful or unsuccessful). This may include authentication attempts from an unusual location or client, partially successful authentication (only one of multiple factors), an authentication attempt after a long period of inactivity or a successful authentication after several unsuccessful attempts.	3
6.3.6	Verify that email is not used as either a single-factor or multi-factor authentication mechanism.	3
6.3.7	Verify that users are notified after updates to authentication details, such as credential resets or modification of the username or email address.	3
6.3.8	Verify that valid users cannot be deduced from failed authentication challenges, such as by basing on error messages, HTTP response codes, or different response times. Registration and forgot password functionality must also have this protection.	3

V6.4 Authentication Factor Lifecycle and Recovery

Authentication factors may include passwords, soft tokens, hardware tokens, and biometric devices. Securely handling the lifecycle of these mechanisms is critical to the security of an application, and this section includes requirements related to this.

The requirements in this section mostly relate to § 5.1.1.2 or § 6.1.2.3 of NIST's Guidance.

#	Description	Level
6.4.1	Verify that system generated initial passwords or activation codes are securely randomly generated, follow the existing password policy, and expire after a short period of time or after they are initially used. These initial secrets must not be permitted to become the long term password.	1
6.4.2	Verify that password hints or knowledge-based authentication (so-called “secret questions”) are not present.	1
6.4.3	Verify that a secure process for resetting a forgotten password is implemented, that does not bypass any enabled multi-factor authentication mechanisms.	2
6.4.4	Verify that if a multi-factor authentication factor is lost, evidence of identity proofing is performed at the same level as during enrollment.	2
6.4.5	Verify that renewal instructions for authentication mechanisms which expire are sent with enough time to be carried out before the old authentication mechanism expires, configuring automated reminders if necessary.	3
6.4.6	Verify that administrative users can initiate the password reset process for the user, but that this does not allow them to change or choose the user’s password. This prevents a situation where they know the user’s password.	3

V6.5 General Multi-factor authentication requirements

This section provides general guidance that will be relevant to various different multi-factor authentication methods.

The mechanisms include:

- Lookup Secrets
- Time based One-time Passwords (TOTPs)
- Out-of-Band mechanisms

Lookup secrets are pre-generated lists of secret codes, similar to Transaction Authorization Numbers (TAN), social media recovery codes, or a grid containing a set of random values. This type of authentication mechanism is considered “something you have” because the codes are deliberately not memorable so will need to be stored somewhere.

Time based One-time Passwords (TOTPs) are physical or soft tokens that display a continually changing pseudo-random one-time challenge. This type of authentication mechanism is considered “something you have”. Multi-factor TOTPs are similar to single-factor TOTPs, but require a valid PIN code, biometric unlocking, USB insertion or NFC pairing, or some additional value (such as transaction signing calculators) to be entered to create the final One-time Password (OTP).

Details on out-of-band mechanisms will be provided in the next section.

The requirements in these sections mostly relate to § 5.1.2, § 5.1.3, § 5.1.4.2, § 5.1.5.2, § 5.2.1, and § 5.2.3 of NIST's Guidance.

#	Description	Level
6.5.1	Verify that lookup secrets, out-of-band authentication requests or codes, and time-based one-time passwords (TOTPs) are only successfully usable once.	2
6.5.2	Verify that, when being stored in the application's backend, lookup secrets with less than 112 bits of entropy (19 random alphanumeric characters or 34 random digits) are hashed with an approved password storage hashing algorithm that incorporates a 32-bit random salt. A standard hash function can be used if the secret has 112 bits of entropy or more.	2
6.5.3	Verify that lookup secrets, out-of-band authentication code, and time-based one-time password seeds, are generated using a Cryptographically Secure Pseudorandom Number Generator (CSPRNG) to avoid predictable values.	2
6.5.4	Verify that lookup secrets and out-of-band authentication codes have a minimum of 20 bits of entropy (typically 4 random alphanumeric characters or 6 random digits is sufficient).	2
6.5.5	Verify that out-of-band authentication requests, codes, or tokens, as well as time-based one-time passwords (TOTPs) have a defined lifetime. Out of band requests must have a maximum lifetime of 10 minutes and for TOTP a maximum lifetime of 30 seconds.	2
6.5.6	Verify that any authentication factor (including physical devices) can be revoked in case of theft or other loss.	3
6.5.7	Verify that biometric authentication mechanisms are only used as secondary factors together with either something you have or something you know.	3
6.5.8	Verify that time-based one-time passwords (TOTPs) are checked based on a time source from a trusted service and not from an untrusted or client provided time.	3

V6.6 Out-of-Band authentication mechanisms

This usually involves the authentication server communicating with a physical device over a secure secondary channel. For example, sending push notifications to mobile devices. This type of authentication mechanism is considered "something you have".

Unsafe out-of-band authentication mechanisms such as e-mail and VOIP are not permitted. PSTN and SMS authentication are currently considered to be "restricted" authentication mechanisms by